Universidad Nacional Autónoma de México

Facultad de Ingeniería

Integrantes:

EGarcia Cortes Adolfo de jesus - 320060829

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320060829

Sistemas distribuidos

Grupo: 01 - Semestre: 2026-2

Proyecto 1:

Caso de estudio Interfaz de paso de mensajes y Cluster HPC

Profesora:

M.I. Elba Karen Saenz Garcia

Fecha de Entrega:

16 de Marzo del 2026

Índice

1 Objetivo: **1**

2 Desarrollo: **1**

2.1 Documentacion del cluster: 1

2.1.1 Actividad 1: 1

2.2 Actividad 2: 2

2.2.1 Programa a), Multiplicacion de matrices: 2

2.2.2 Programa b), Determinacion de renglos de matriz que tiene la mayor suma de sus elementos: 3

3 Tabla de participacion en el proyecto: **13**

4 Conclusiones: **14**

Referencias **15**

1. Objetivo:

Construir un sistema distribuido de cómputo de alto rendimiento (HPC) y ejecutar programas paralelos que distribuyan el trabajo entre los nodos que lo conforman.

2. Desarrollo:

Para realizacion del proyecto se tiene como funciones disponibles de MPI las siguientes:

- **MPI_Init:** Inicializa el entorno de ejecución de MPI.
- **MPI_Comm_size:** Determina el número total de procesos en el comunicador.
- **MPI_Comm_rank:** Determina el identificador único (rango) del proceso dentro del comunicador.
- **MPI_Send:** Envía un mensaje a otro proceso.
- **MPI_Recv:** Recibe un mensaje de otro proceso.
- **MPI_Finalize:** Finaliza el entorno de ejecución de MPI.
- **MPI_Gather:** Recopila datos de todos los procesos en el comunicador.
- **MPI_Scatter:** Distribuye datos desde un proceso raíz a todos los procesos en el comunicador.
- **MPI_Bcast:** Difunde un mensaje desde un proceso raíz a todos los procesos en el comunicador.
- **MPI_Reduce:** Realiza una operación de reducción (como suma, máximo, etc.) en datos distribuidos entre los procesos y devuelve el resultado al proceso raíz.

De las cuales, para el desarrollo de los programas, solo se usaron las funciones **MPI_Init**, **MPI_Comm_size**, **MPI_Comm_rank**, **MPI_Send**, **MPI_Recv**, **MPI_Scatter**, **MPI_Gather** y **MPI_Finalize**.

2.1. Documentacion del cluster:

2.1.1. Actividad 1:

2.2. Actividad 2:

2.2.1. Programa a), Multiplicacion de matrices:

Codigo fuente (C):

2.2.2. Programa b), Determinacion de renglos de matriz que tiene la mayor suma de sus elementos:

Enunciado: Se tiene una matriz A de tamaño $N \times M$ que contiene números enteros. Desarrollar un programa distribuido utilizando MPI que determine qué renglón de la matriz tiene la mayor suma de sus elementos.

El programa deberá distribuir el trabajo entre varios procesos de manera que cada proceso analice una parte de la matriz.

El proceso 0 debe generar o inicializar la matriz A; la matriz deberá distribuirse entre los procesos utilizando comunicaciones colectivas de MPI. Cada proceso calculará la suma de los elementos de los renglones que le correspondan. Cada proceso determinará la fila con mayor suma dentro de su bloque local.

El proceso 0 deberá imprimir: el índice de la fila con mayor suma y el valor de dicha suma.

Suponer que N es divisible entre el número de procesos.

Resolucion : Al iniciar el programa tenemos las siguientes lineas:

Listing 1: Generacion de matriz por el proceso 0 e impresion

```
int main(int argc, char **argv) {
int np, id;
int *A = NULL;
int *local_A;
int rows_per_proc;
char hostname[256];

MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &np);
MPI_Comm_rank(MPI_COMM_WORLD, &id);
gethostname(hostname, sizeof(hostname));

rows_per_proc = N / np;

if (id == 0 && N % np != 0) {
    printf("Error: N (%d) debe ser divisible entre np (%d)\n", N, np);
    MPI_Finalize();
    return 1;
}
```

Primero se declaran las variables necesarias para el programa, como el número de procesos (`np`), el identificador del proceso (`id`), un puntero para la matriz A y un puntero para el bloque local de cada proceso (`local_A`), una variable para almacenar el número de filas que cada proceso analizará (`rows_per_proc`) y un arreglo para almacenar el nombre del host donde se ejecuta cada proceso.

Luego se inicializa el entorno de ejecución de MPI con `MPI_Init()`, se obtiene el número total de procesos con `MPI_Comm_size()` y el identificador único de cada proceso con `MPI_Comm_rank()`. También se obtiene el nombre del host utilizando la función `gethostname()`.

Después de esto, se calcula el número de filas que cada proceso analizará dividiendo el número total de filas N entre el número de procesos `np`. Se verifica que N sea divisible entre `np`, y si no lo es, el proceso 0 imprime un mensaje de error y finaliza el programa.

Para la creacion de este programa, primero definimos el tamaño de la matriz $N \times M$ y el número de procesos a utilizar.

Para esto el tamaño de N, es de 8 y el tamaño de M es de 5, el proceso 0 se encarga de generar la matriz A con números enteros aleatorios.

Fragmento de codigo de generacion de matriz (C):

Listing 2: Generacion de matriz por el proceso 0 e impresion

```c
    if (id == 0) {
        A = (int *)malloc(N * M * sizeof(int));
        srand(42);
        for (int i = 0; i < N * M; i++)
            A[i] = rand() % 100;

        separador();
        printf("  FILA CON MAYOR SUMA - %d procesos, Matriz %dx%d\n", np, N, M
            );
        separador();
        printf("\n  [P0] Nodo: %s | Genera la matriz A\n\n", hostname);
        print_matrix(A, N, M);
        separador();
        printf("  [P0] Nodo: %s | Distribuyendo %d filas a cada proceso (
            MPI_Scatter)\n", hostname, rows_per_proc);
        separador();
    }
```

Para esto, cuando la condicion evalua al proceso 0 se encarga de generar la matriz A con números enteros aleatorios, utilizando la función `rand()` y una semilla fija. Despues de esto, el proceso 0 imprime la matriz generada, llamando a la funcion `print_matrix()`, la cual para imprimir la matriz, hace uso de un ciclo anidado para recorrer los elementos de la matriz y mostrar los elementos de la misma en formato de filas y columnas.

Listing 3: Funcion de impresion de matriz

```c
void print_matrix(int *mat, int rows, int cols) {
    printf("          ");
    for (int j = 0; j < cols; j++)
        printf("Col%-3d", j);
    printf("\n");
    for (int i = 0; i < rows; i++) {
        printf("  Fila%2d [", i);
        for (int j = 0; j < cols; j++) {
            printf("%4d", mat[i * cols + j]);
            if (j < cols - 1) printf(", ");
        }
        printf(" ]\n");
    }
}
```

Primero se reserva memoria para el bloque local de cada proceso, que es un bloque de la matriz A con un número de filas igual a `rows_per_proc` y el mismo número de columnas M.

Despues de esto, el proceso 0 distribuye la matriz A entre los procesos utilizando la función `MPI_Scatter()`, la cual se encarga de enviar partes de la matriz a cada proceso, para esto los elementos de esta funcion se componen:

Listing 4: Distribucion de matriz con MPI_Scatter

```c
    local_A = (int *)malloc(rows_per_proc * M * sizeof(int));
    MPI_Scatter(A, rows_per_proc * M, MPI_INT, local_A, rows_per_proc * M,
        MPI_INT, 0, MPI_COMM_WORLD);
```

Donde el primer argumento es el puntero a la matriz A que se va a distribuir, el segundo argumento es el número de elementos que cada proceso recibirá (en este caso, el número de filas por proceso multiplicado por el número de columnas), el tercer argumento especifica el tipo de dato (en este caso, enteros), el cuarto argumento es un puntero al bloque local donde cada proceso almacenará su parte de la matriz, el quinto argumento es nuevamente el número de elementos que cada proceso recibirá, el sexto argumento especifica el tipo de dato para la recepción, el séptimo argumento indica el proceso raíz (en este caso, 0) y el octavo argumento es el comunicador.

Ahora vien sepues de estos e empieza con el calculo de la suma, para esto primero inicializaciamos las variables `fila_global_inicio = id * rows_per_proc;`, la cual se encarga de calcular el índice de la primera fila que cada proceso analizará, tambien tenemos a las variables `local_max_sum` y `local_max_row`, las cuales se encargan de almacenar la suma máxima local y el índice de la fila con la suma máxima local, respectivamente.

Despues de esto se imprime que procesos recibio que pila en especifico, donde la linea de codigo es la siguiente:

Listing 5: Impresion de bloque local recibido por cada proceso

```
        printf("\n  [P%d] Nodo: %s | Recibio filas %d a %d\n",
           id, hostname, fila_global_inicio, fila_global_inicio +
              rows_per_proc - 1);
```

Luego de esto, ahora los procesos lo que haran sera entrar en ciclo for para calcular la suma de cada fila dentro de su bloque local, y determinar la fila con la suma máxima local.

Para esto, el bloque de codigo es el siguiente:

Listing 6: Calculo de suma de filas y determinacion de fila con suma maxima local

```
        for (int i = 0; i < rows_per_proc; i++) {
            int sum = 0;
            int fila_global = fila_global_inicio + i;

            printf("  [P%d] Nodo: %s | Sumando fila %d: ", id, hostname,
               fila_global);
            for (int j = 0; j < M; j++) {
                sum += local_A[i * M + j];
                if (j > 0) printf(" + ");
                printf("%d", local_A[i * M + j]);
            }
            printf(" = %d\n", sum);

            if (i == 0 || sum > local_max_sum) {
                local_max_sum = sum;
                local_max_row = i;
            }
        }
```

En este bloque de codigo, cada proceso recorre las filas de su bloque local, calcula la suma de los elementos de cada fila y determina cuál es la fila con la suma máxima local. Para esto, se utiliza un ciclo for para iterar sobre las filas del bloque local, y dentro de este ciclo, se calcula la suma de los elementos de cada fila utilizando otro ciclo for anidado.

La estructura de funcionamiento del ciclo for anidado, es por medio de un contador de filas locales, el cual se encarga de recorrer las filas del bloque local, y dentro de este ciclo, se tiene otro ciclo for que recorre las columnas de la fila actual para calcular la suma de los elementos de esa fila.

Ademas, se imprime el proceso que esta realizando la suma, el nodo en el que se encuentra, la fila que esta sumando y los elementos de esa fila, junto con el resultado de la suma. Luego de esto, se compara la suma calculada con la suma máxima local almacenada en la variable `local_max_sum`, y si la suma actual es mayor, se actualiza la suma máxima local y el índice de la fila con la suma máxima local.

Siguiendo con las demas linea de codigo, tenemos la inicializacion de la variables `global_max_row = fila_global_inicio + local_max_row;`, donde se calcula el índice global de la fila con la suma máxima local, sumando el índice de la primera fila del bloque local con el índice de la fila con la suma máxima local dentro de ese bloque.

Luego de esto, se imprime el resultado local de cada proceso, indicando cuál es la fila con la suma máxima local y el valor de esa suma.

Listing 7: Impresion de resultado local de cada proceso

```
        int global_max_row = fila_global_inicio + local_max_row;

        printf("  [P%d] Nodo: %s | Mayor suma local -> Fila %d (suma = %d)\n",
            id, hostname, global_max_row, local_max_sum);
        separador();
```

Despues de esto, el proceso 0 se encarga de reservar memoria para almacenar los resultados locales de cada proceso, que consisten en el índice global de la fila con la suma máxima local y el valor de esa suma máxima local. Para esto, el bloque de codigo es el siguiente:

Listing 8: Reserva de memoria para resultados locales en proceso 0

```
        int *local_max_rows = NULL;
        int *local_max_sums = NULL;
        if (id == 0) {
            local_max_rows = (int *)malloc(np * sizeof(int));
            local_max_sums = (int *)malloc(np * sizeof(int));
        }
```

En seguida, se utiliza la función `MPI_Gather()` para recopilar los resultados locales de cada proceso en el proceso 0.

Para esto, el bloque de codigo es el siguiente:

Listing 9: Recopilacion de resultados locales con MPI_Gather

```
        MPI_Gather(&global_max_row, 1, MPI_INT, local_max_rows, 1, MPI_INT, 0,
            MPI_COMM_WORLD);
```

```
        MPI_Gather(&local_max_sum, 1, MPI_INT, local_max_sums, 1, MPI_INT, 0,
            MPI_COMM_WORLD);
```

Para la primera linea de `MPI_Gather()`, tenemos que el primer argumento es un puntero al dato local que cada proceso va a enviar (en este caso, el índice global de la fila con la suma máxima local), el segundo argumento es el número de elementos que cada proceso enviará (en este caso, 1), el tercer argumento especifica el tipo de dato (en este caso, enteros), el cuarto argumento es un puntero al arreglo donde el proceso raíz almacenará los datos recopilados (en este caso, `local_max_rows`), el quinto argumento es nuevamente el número de elementos que cada proceso enviará, el sexto argumento especifica el tipo de dato para la recepción, el séptimo argumento indica el proceso raíz (en este caso, 0) y el octavo argumento es el comunicador. La segunda linea de `MPI_Gather()` es similar, pero en este caso se recopilan los valores de las sumas máximas locales en el arreglo `local_max_sums`.

Por ultimo tenemos el final del programa, donde por medio de una condicional se veririfica que el que imprima el resultado y calcula la ultima parte sea el procesos 0, donde al entrar a la conficional, el proceso 0 imrpimira los resiltados e imprimira un anuncio que el proceo 0 recolecto los resultados y luego de esto, se entra en un ciclo for para determinar cuál es la fila con la suma máxima global, comparando las sumas máximas locales recopiladas y sus correspondientes índices de fila.

Finalmente, el proceso 0 imprime el resultado final, indicando cuál es la fila con la suma máxima global y el valor de esa suma, esto lo hace por medio de la siguiente linea de codigo:

Listing 10: Impresion de resultado final en proceso 0

```
        if (id == 0) {
            printf("\n");
            separador();
            printf("  RESULTADO FINAL\n");
            separador();
            printf("  [P0] Nodo: %s | Recolecto resultados de %d procesos (
                MPI_Gather)\n\n", hostname, np);

            int best_sum = all_sums[0];
            int best_row = all_rows[0];

            for (int i = 0; i < np; i++) {
                printf("  -> P%d reporta: Fila %d con suma %d", i, all_rows[i
                    ], all_sums[i]);
                if (i == 0 || all_sums[i] > best_sum) {
                    if (i > 0) {
                        best_sum = all_sums[i];
```

```
                        best_row = all_rows[i];
                    }
                }
                printf("\n");
            }

            printf("\n  >>> La fila con mayor suma es la FILA %d con suma = %d <<<\n", best_row, best_sum);
            separador();

            free(all_sums);
            free(all_rows);
        }
```

La conficional dentro del ciclo for se encarga de comparar las sumas máximas locales recopiladas y actualizar la suma máxima global y el índice de la fila con la suma máxima global si se encuentra una suma mayor, el orden de impresion es por medio de un ciclo for que recorre los resultados recopilados y los imprime, indicando qué proceso reportó cada fila y su suma correspondiente. Finalmente, se imprime el resultado final indicando cuál es la fila con la suma máxima global y el valor de esa suma, esta se almaceno en las variables `best_row` y `best_sum` respectivamente. Finalmente el proceso 0 libera la memoria asignada para los arreglos de resultados locales.

Resultados:

Al contruir el cluster, primero se verifico que se tuvieran el numero de clientes indicados como minimo que fueron 2, por medio del comando:

```
        cat /etc/hosts
```

Donde la salida fue la siguiente:

```
        user@pel1:~$ cat /etc/hosts
        127.0.0.1   localhost
        10.11.12.1  pel1
        # The following lines are desirable for IPv6 capable hosts
        ::1     localhost ip6-localhost ip6-loopback
        fe00::0 ip6-localnet
        ff00::0 ip6-mcastprefix
        ff02::1 ip6-allnodes
        ff02::2 ip6-allrouters
        ff02::3 ip6-allhosts
        10.11.12.19 pel19
```

```
10.11.12.35 pel35
```

Donde se puede observar que se tienen los clientes pel19 y pel35, por lo que se tiene el numero de clientes necesarios para la realizacion del proyecto.

Despues de esto, se crea el programa por medio de la instruccion nano:

```
user@pel1:~$ nano proyecto_b.c
```

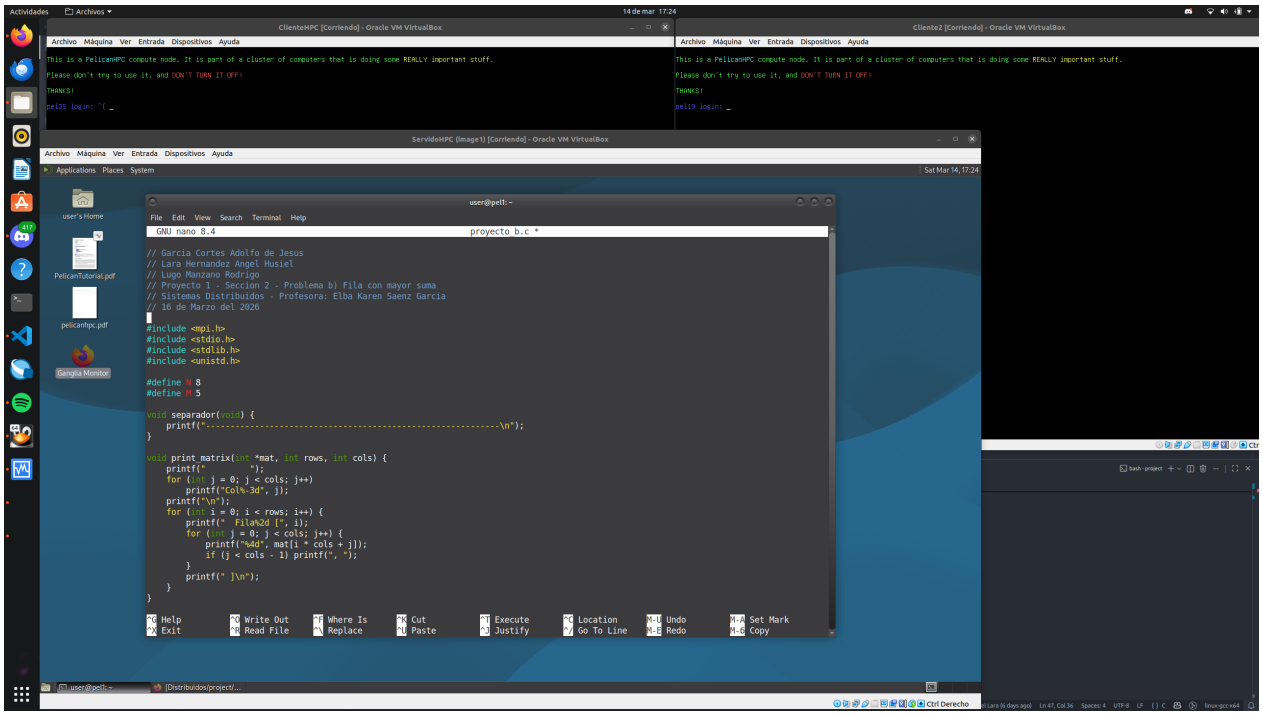

Figura 1: Se crea el programa b) utilizando el editor de texto nano

compilamos el programa es por medio del siguiente comando:

Listing 11: Compilacion de programa b)

```
mpicc proyecto_b.c -o proyecto
```

Y para ejecutar el programa, se utiliza el siguiente comando:

Listing 12: Ejecucion de programa b)

```
mpirun -np 4 ./proyecto
```

Las evidencias son las siguientes:

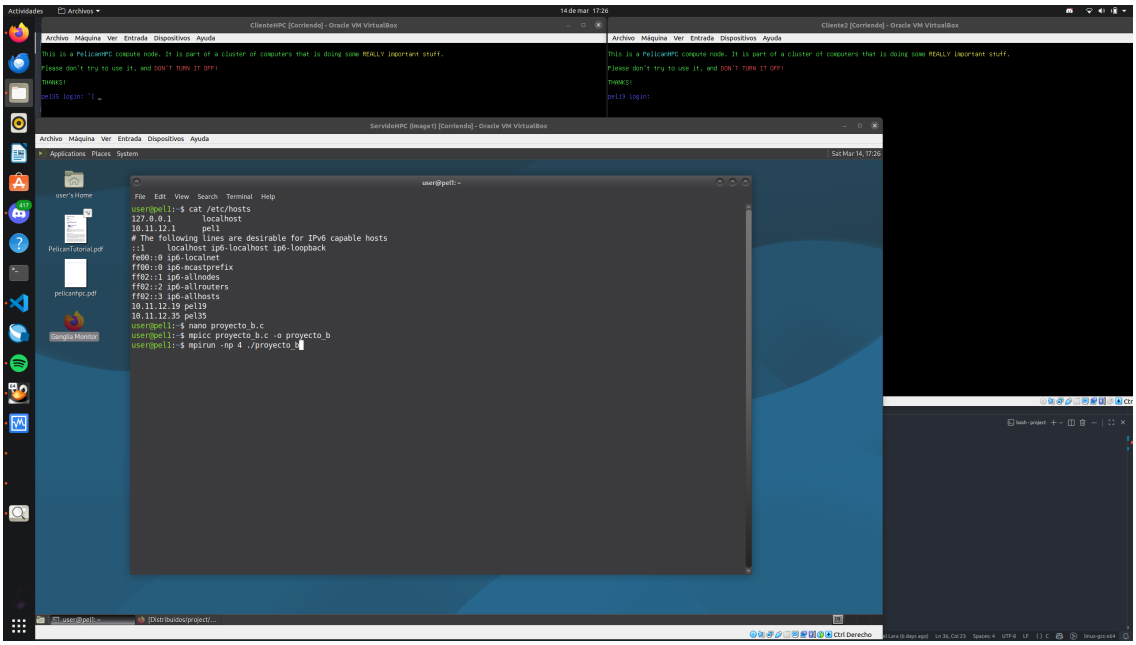

Figura 2: Evidencia de ejecucion del programa b) con 4 procesos

Ahora bien, compilando el programa se puede observar la siguiente salida que estan divida en dos fotos:

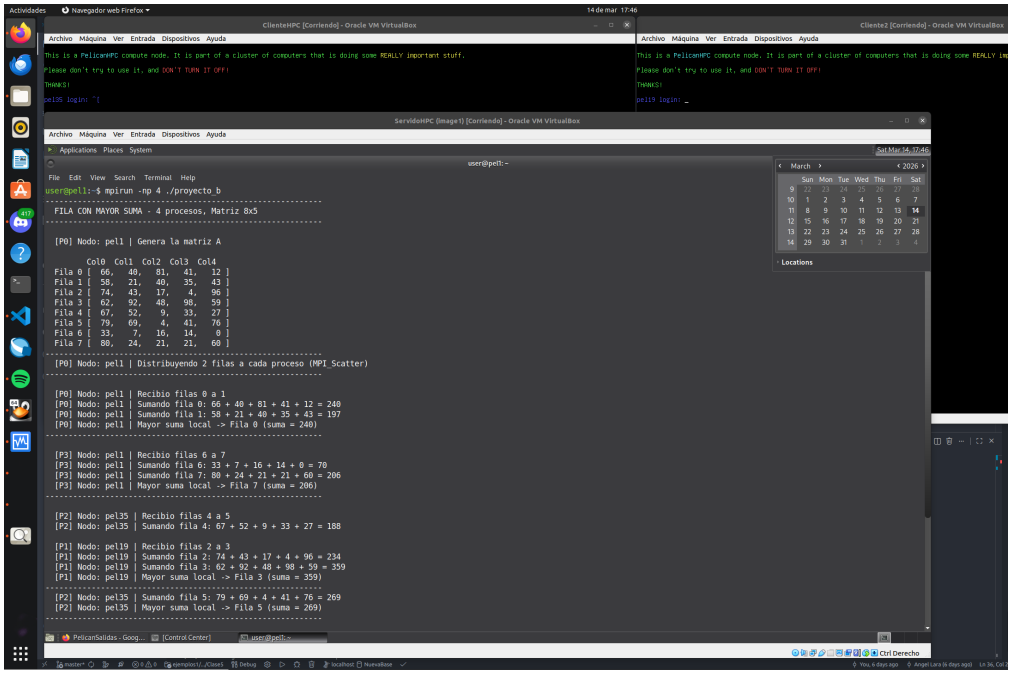

Figura 3: Evidencia de ejecucion del programa b) con 4 procesos (parte 1)

Se ve como la matriz que fue generada por el proceso 0 es la siguiente:

```
        [P0] Nodo: pel1 | Genera la matriz A

                Col0  Col1  Col2  Col3  Col4
        Fila 0 [  66,   40,   81,   41,   12 ]
        Fila 1 [  58,   21,   40,   35,   43 ]
        Fila 2 [  74,   43,   17,   4,   96 ]
        Fila 3 [  62,   92,   48,   98,   59 ]
        Fila 4 [  67,   52, 9,   33,   27 ]
        Fila 5 [  79,   69, 4,   41,   76 ]
        Fila 6 [  33,   7,   16,   14,   0 ]
        Fila 7 [  80,   24,   21,   21,   60 ]
```

Despues de esto, se ve como los clientes reciben cada seccion de la matriz cada uno respectivamente.

```
        [P0] Nodo: pel1 | Recibio filas 0 a 1
        [P1] Nodo: pel19 | Recibio filas 2 a 3
        [P2] Nodo: pel35 | Recibio filas 4 a 5
```

Seguidamente, se tiene la segunda parte de la salida del programa, que termina con la salida del resultado final, donde se puede observar que la mayor suma es de 359:

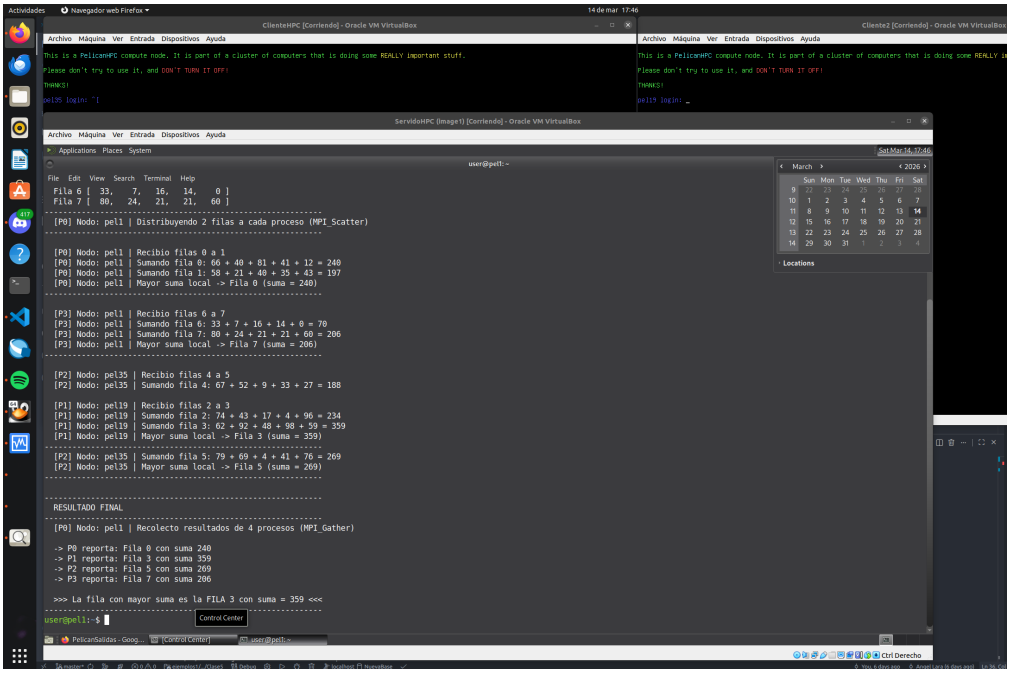

Figura 4: Evidencia de ejecucion del programa b) con 4 procesos (parte 2)

La salida muestra que el proceso 0 genera la matriz de 8×5 y la reparte en bloques de 2 filas por proceso mediante `MPI_Scatter`. Posteriormente, cada proceso calcula la suma de sus filas locales y reporta su mejor resultado: P0 obtiene fila 0 con 240, P1 obtiene fila 3 con 359, P2 obtiene fila 5 con 269 y P3 obtiene fila 7 con 206. Finalmente, el proceso 0 recopila esos resultados con `MPI_Gather`, compara las sumas locales y determina correctamente que la fila con mayor suma global es la **fila 3** con un valor de **359**.

3. Tabla de participacion en el proyecto:

Integrante	**Participacion**
Garcia Cortes Adolfo de Jesus	
Lara Hernandez Angel Husiel	
Lugo Manzano Rodrigo	

Tabla 1: Participacion de integrantes en el proyecto

4. Conclusiones:

- **Lugo Manzano Rodrigo:**
- **Garcia Cortes Adolfo de jesus:**
- **Lara Hernandez Angel Husiel:**

Referencias

Anaya, R. (s.f.). *Manual de recursos y aplicaciones Plataforma Rasberry Pi*. https://odin.fi-b.unam.mx/micros/docs/Tutoriales%20Raspberry.pdf

Elahi, A. (2022, 17 de marzo). *Computer systems: Digital Design, Fundamentals of Computer Architecture and ARM Assembly Language* (2.ª ed.). Springer. https://doi.org/10.1007/978-3-030-93449-1

Harris, D., & Harris, S. (2015, 22 de abril). *Digital Design and Computer Architecture* (Arm Edition). Morgan Kaufmann Pub.

Smith, S. (2019, octubre). *Raspberry Pi Assembly Language Programming*. Apress. https://doi.org/10.1007/978-1-4842-5287-1